

Chương 6: Ngữ nghĩa và Phân tích quan điểm

Phân tích và Khai phá dữ liệu văn bản

TS. Nguyễn Thị Mai Trang

Sử dụng WordNet

6.1.1. Synsets – Khái niệm

Synset (Synonym Set): Là một tập hợp các từ đồng nghĩa (synonyms).

Đơn vị trung tâm của WordNet là **Synset**

Mỗi synset đại diện cho một khái niệm ngữ nghĩa duy nhất, nhóm các từ có nghĩa tương tự lại với nhau.

- **Ví dụ:** Một synset có thể chứa: {car, auto, automobile, machine, motorcar}.
- Chúng đều chỉ chung một khái niệm về "xe hơi".
- WordNet chứa hơn 117,000 synset.

6.1.1. Synsets – Tính đa nghĩa (Polysemy)

Ví dụ về Tính đa nghĩa

Một từ (word form) có thể xuất hiện trong nhiều synset khác nhau, tùy thuộc vào nghĩa của nó trong ngữ cảnh.

Từ: **bank**

- ✓ Synset('bank.n.02'): (Tổ chức tài chính – "financial institution")
- ✓ Synset('bank.n.01'): (Bờ sông – "sloping land beside a river")
- ✓ Synset('bank.v.01'): (Hành động gửi tiền – "to deposit money")

```
# Truy cập trong NLTK (Python)
from nltk.corpus import wordnet as wn

# Tìm các synset của 'bank'
wn.synsets('bank')
#> [Synset('bank.n.01'),
# Synset('bank.n.02'), ...]

# Lấy định nghĩa
wn.synset('bank.n.02').definition()
```

```
[nltk_data] Downloading package wordnet
[Synset('bank.n.01'),
Synset('depository_financial_institution'),
Synset('bank.n.03'),
Synset('bank.n.04'),
Synset('bank.n.05'),
Synset('bank.n.06'),
Synset('bank.n.07'),
Synset('savings_bank.n.02'),
Synset('bank.n.09'),
Synset('bank.n.10'),
Synset('bank.v.01'),
Synset('bank.v.02'),
Synset('bank.v.03'),
Synset('bank.v.04'),
```

```
print(wn.synset('bank.n.01').definition())
print(wn.synset('bank.n.02').definition())
```

sloping land (especially the slope beside a body of water)
a financial institution that accepts deposits and channels the money i

6.1.1. Synsets – Ứng dụng thực tế



Mở rộng Truy vấn (Query Expansion)

Khi người dùng tìm kiếm "car", hệ thống cũng có thể tìm các tài liệu chứa "automobile" hoặc "motorcar".



Tạo dữ liệu (Data Augmentation)

Tự động tạo thêm dữ liệu huấn luyện bằng cách thay thế một từ trong câu bằng từ đồng nghĩa của nó từ synset.



Đo lường tương tự (Similarity)

Synsets là nền tảng để tính toán độ tương tự ngữ nghĩa giữa các từ (ví dụ: 'car' gần với 'truck' hơn là 'car' với 'apple').

6.1.2 Phân tích quan hệ ngữ nghĩa

WordNet không chỉ chứa các từ, mà còn liên kết các synset qua các quan hệ ngữ nghĩa:

- **Hypernymy / Hyponymy (Bao nghĩa / Hạ nghĩa):** Mỗi quan hệ "là một loại của".
*Ví dụ: 'dog' (chó) là **hyponym** (hạ nghĩa) của 'canine' (họ chó).*
- **Meronymy / Holonymy (Toàn thể / Bộ phận):** Mỗi quan hệ "là một phần của".
*Ví dụ: 'wheel' (bánh xe) là **meronym** (bộ phận) của 'car' (xe hơi).*
- **Antonymy (Trái nghĩa):** Liên kết các từ có nghĩa đối lập.
*Ví dụ: 'hot' (nóng) là **antonym** của 'cold' (lạnh).*

6.1.2. Quan hệ: Hypernyms & Hyponyms (IS-A)

Hypernym là từ bao hàm (hay từ tổng quát hơn) so với các từ khác.

Nó mô tả một lớp hoặc nhóm khái niệm rộng mà các từ cụ thể hơn thuộc vào.

Ví dụ:

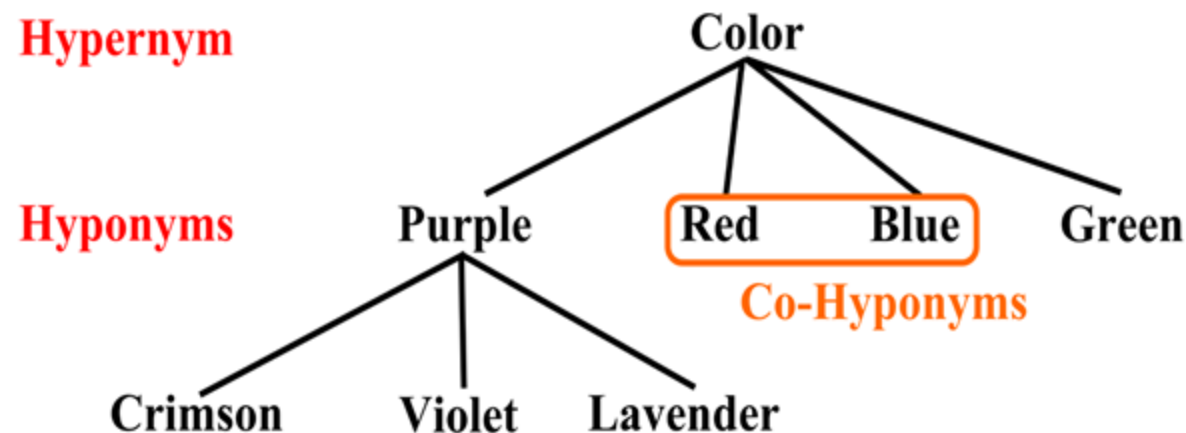
- “Animal” là hypernym của “dog”, “cat”, “bird”.
- “Vehicle” là hypernym của “car”, “bus”, “motorbike”.

Hyponym là từ bị bao hàm (từ cụ thể hơn) trong mối quan hệ với một từ tổng quát hơn (hypernym).

Ví dụ:

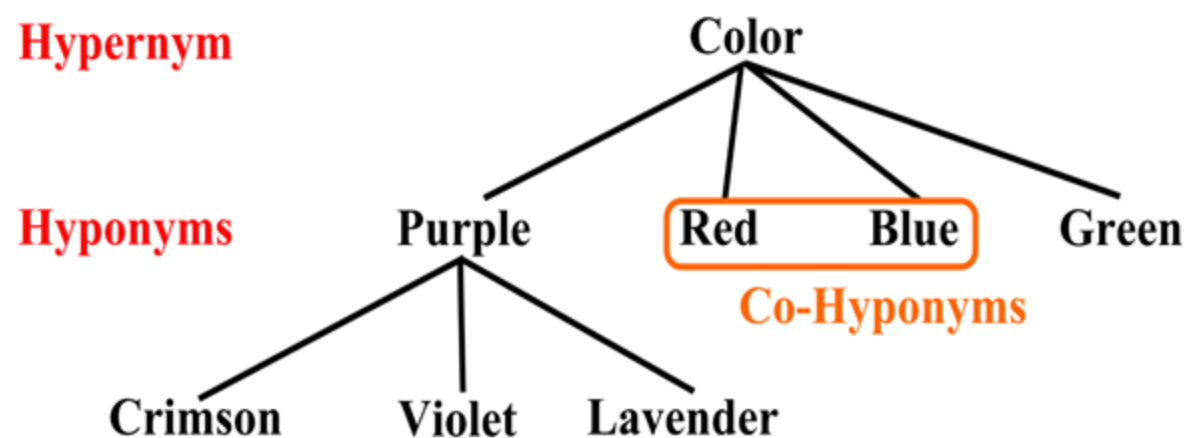
- “Dog” là hyponym của “animal”.
- “Car” là hyponym của “vehicle”.

Nếu A là **hypernym** của B, thì B là **hyponym** của A.



6.1.2. Quan hệ: Hypernyms & Hyponyms (IS-A)

- **Mối quan hệ giữa Hypernym và Hyponym:** Quan hệ “is-a” (là một loại của):
 - **animal** → hypernym của **dog**, **cat**, **bird**
 - **dog** → hyponym của **animal**
 - **bulldog** → hyponym của **dog** và gián tiếp của **animal**



6.1.2. Quan hệ: Meronyms & Holonyms (PART-OF)



Quan hệ **"PART-OF"** (Là-một-phần-của): Mô

tả mối quan hệ bộ phận/tổng thể.



Meronym (Bộ phận): Là một phần của một tổng thể.

Ví dụ: wheel (bánh xe) là meronym của car.



Holonym (Tổng thể): Chứa hoặc có các bộ phận.

Ví dụ: car là holonym của wheel.



Ý nghĩa thực tế: Giúp máy tính hiểu cấu trúc

vật lý hoặc thành phần của các đối tượng (ví

dụ: một 'car' có 'engine' và 'wheels').

6.1.2. Ứng dụng của các quan hệ ngữ nghĩa



Đo lường tương tự (Similarity)

Tính toán "khoảng cách" giữa hai synset trong cây hypernym. **Ví dụ:** 'dog' và 'cat' gần nhau (vì cùng có hypernym là 'carnivore') hơn là 'dog' và 'tree'.



Suy luận (Reasoning)

Nếu A là hyponym của B, và B là hyponym của C, thì A cũng mang đặc điểm của C. (Ví dụ: 'poodle' -> 'dog' -> 'animal').



Phân cụm văn bản

Nhóm các tài liệu nói về "bác sĩ", "y tá", "bệnh viện" vào cùng một cụm "Y tế" (hypernym) thay vì coi chúng là các từ riêng biệt.

6.1.3. WSD – Word Sense Disambiguation

Bài toán WSD

WSD là nhiệm vụ xác định nghĩa nào của một từ (word sense) đang được sử dụng trong một ngữ cảnh cụ thể.

Ví dụ: "bank"

✓ "I went to the **bank** to deposit money."

→ (Nghĩa: *Tổ chức tài chính*)

✓ "He sat on the **bank** of the river."

→ (Nghĩa: *Bờ sông*)

WordNet cung cấp một kho lưu trữ các nghĩa có thể có để máy tính tham chiếu.

Tại sao WSD quan trọng?

Máy tính không thể hiểu chính xác một câu nếu không hiểu đúng nghĩa của các từ trong đó.

Ý nghĩa thực tế: WSD là bước thiết yếu cho Dịch máy, Tìm kiếm thông tin và Phân tích quan điểm. Nếu dịch sai "bank" (Anh) sang tiếng Pháp (thành 'banque' hay 'rive?'), câu sẽ sai nghĩa hoàn toàn.

6.1.3. WSD – Phương pháp Lesk (Knowledge-based)

 **Phương pháp dựa trên Từ điển/Tri thức (Knowledge-based).**

 **Ý tưởng cốt lõi:** Nghĩa (synset) chính xác là nghĩa có định nghĩa (gloss) giống với ngữ cảnh xung quanh (context) nhất.

 **Cách hoạt động (Đơn giản):**

1. Lấy các từ trong ngữ cảnh của từ mục tiêu (ví dụ: "river", "sat" cho "bank").
2. Với mỗi synset của "bank" (vd: 'bank.n.01', 'bank.n.02'), lấy định nghĩa (gloss) của nó.
3. Đếm số từ *trùng lặp* (overlap) giữa (1) và (2).
4. Chọn synset có điểm overlap cao nhất.

 **Ưu điểm:** Đơn giản, không cần dữ liệu huấn luyện đã gán nhãn.

6.1.3. WSD – Phương pháp có giám sát



Xem WSD như bài toán Phân loại (Classification).



Dữ liệu huấn luyện: Cần một kho văn bản đã được gán nhãn nghĩa (Sense-annotated corpus), ví dụ: SemCor.



Trích chọn đặc trưng (Features):

Các từ xung quanh (context words) (ví dụ: "river", "deposit", "money").

Từ loại (POS tags) của từ mục tiêu và các từ xung quanh.



Mô hình (Classifier): Sử dụng các thuật toán như Naive Bayes, SVM, hoặc Mạng nơ-ron.



Ý nghĩa thực tế: Thường cho độ chính xác cao hơn phương pháp Lesk, nhưng đòi hỏi dữ liệu gán nhãn rất tốn kém.

6.1.3. WSD – Ứng dụng thực tế



Dịch máy (Machine Translation)

Để dịch "I'm feeling blue", máy cần biết 'blue' là "buồn" (cảm xúc) chứ không phải "màu xanh" (màu sắc).



Tìm kiếm (Information Retrieval)

Khi tìm kiếm "bass", người dùng muốn kết quả về "loài cá" hay "nhạc cụ"? WSD giúp phân biệt dựa trên các từ khóa khác trong truy vấn.



Phân tích quan điểm

Từ "lạnh lùng" (cool) có thể mang nghĩa tích cực ("That car is cool") hoặc tiêu cực ("He gave me a cool look").

6.1.4. NER – Named Entity Recognition



Khái niệm: Là quá trình xác định (identify) và phân loại (classify) các "thực thể có tên" (Named Entities - NER) trong văn bản.



Thực thể là gì? Là các đối tượng trong thế giới thực có tên riêng, ví dụ: tên người, tên công ty, địa điểm, ngày tháng, tiền tệ...



Các loại (Tags) phổ biến:

PER (Person): "Nguyễn Văn A", "Steve Jobs"

ORG (Organization): "Google", "Đại học Bách Khoa"

LOC (Location): "Hà Nội", "Việt Nam", "Cupertino"

DATE, TIME, MONEY, PRODUCT...



Ý nghĩa thực tế: Giúp máy tính chuyển văn bản phi cấu trúc (unstructured text) thành thông tin có cấu trúc (structured data).

6.1.4. NER – Ví dụ minh họa

Trích xuất thông tin có cấu trúc

NER "đọc" câu và gán nhãn cho các thực thể quan trọng.

Câu gốc: "Apple, led by Tim Cook, is looking at buying a U.K. startup for \$1 billion."

Kết quả NER:

"[**ORG** Apple], led by [**PER** Tim Cook], is looking at buying a [**LOC** U.K.] startup for [\$1 billion **MONEY**]."

6.1.4. NER – Các phương pháp tiếp cận



Dựa trên luật (Rule-based) & Danh sách (Gazetteers):

Sử dụng các mẫu (ví dụ: Regex cho DATE, MONEY).

Sử dụng danh sách (gazetteer) tên các quốc gia, thành phố để nhận diện LOC.

Nhanh, nhưng khó bảo trì và mở rộng.



Học máy cổ điển (Machine Learning):

Conditional Random Fields (CRF): Thuật toán rất phổ biến và hiệu quả cho NER.

Xem NER như bài toán gán nhãn chuỗi (sequence labeling).



Học sâu (Deep Learning) - (State-of-the-art):

BiLSTM-CRF: Kết hợp mạng LSTM hai chiều (nhắm bắt ngữ cảnh) và CRF (tối ưu nhãn).

Transformers (BERT): Sử dụng BERT (hoặc các biến thể) làm lớp trích chọn đặc trưng, cho kết quả SOTA.

6.1.4. NER – Ứng dụng thực tế



Lọc CV (Resume Parsing)

Tự động trích xuất Tên (PER), Trường học (ORG), Kỹ năng (SKILL), Kinh nghiệm (DATE) từ CV để nhập vào cơ sở dữ liệu.



Hệ thống Hỏi-Đáp (QA)

Trả lời câu hỏi: "Ai là CEO của Apple?"
Hệ thống tìm `PER` trong các câu chứa "CEO" và "Apple" (ORG).



Tổng hợp tin tức

Phân tích hàng ngàn bài báo để tìm các công ty (ORG) và cá nhân (PER) được nhắc đến nhiều nhất trong ngày.

Phân tích Biểu diễn Ngữ nghĩa

6.2. Phân tích Vai trò Ngữ nghĩa (SRL)

Hiểu "Ai đã làm gì?"

Semantic Role Labeling (SRL) là quá trình gán nhãn "vai trò" cho các thành phần trong câu, xoay quanh một vị ngữ (thường là động từ).

Ví dụ: "Yesterday, [AGENT John] [PREDICATE broke] [THEME the window] [TIME yesterday]."

- ✓ AGENT: Tác nhân thực hiện (John)
- ✓ THEME: Đối tượng bị tác động (the window)
- ✓ TIME: Thời gian (yesterday)

Ý nghĩa: Cung cấp một biểu diễn "ai làm gì cái gì" (who-did-what-to-whom) cho máy tính.



6.2. Biểu diễn phân bố (Word Embeddings)

Biểu diễn từ bằng Vector

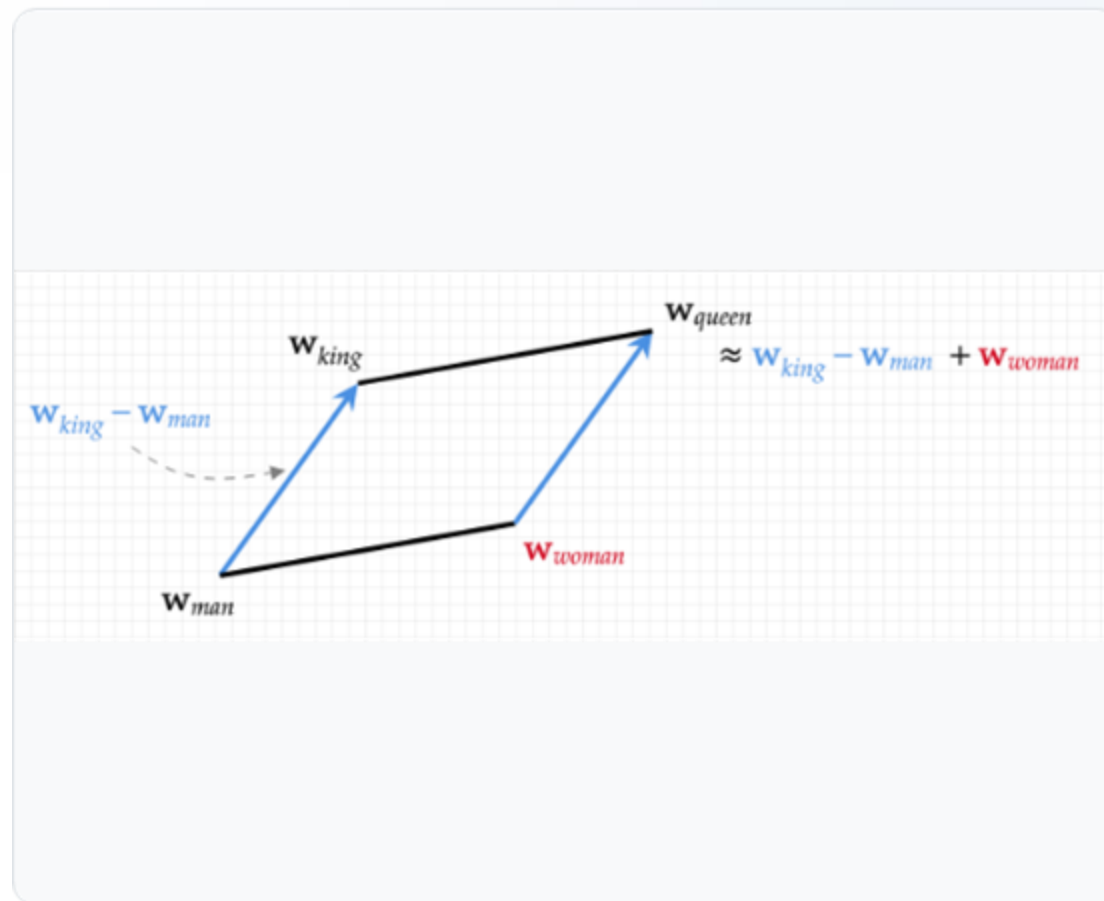
Thay vì xem từ là các đơn vị rời rạc (như trong WordNet), **Word Embeddings** (ví dụ: Word2Vec, GloVe) biểu diễn mỗi từ bằng một vector số thực (ví dụ: 300 chiều).

- ✓ Các từ có nghĩa tương tự sẽ có vector **gần nhau** trong không gian.
- ✓ Nắm bắt được các quan hệ ngữ nghĩa và tương đồng.

Ví dụ kinh điển (Đại số Vector):

$\text{vector}(\text{'King'}) - \text{vector}(\text{'Man'}) + \text{vector}(\text{'Woman'}) \approx \text{vector}(\text{'Queen'})$

Ý nghĩa: Chuyển ngôn ngữ thành dạng toán học mà máy tính có thể học được.



6.2. Biểu diễn theo ngữ cảnh (Contextual)

Vấn đề của Word2Vec

Word2Vec (hay GloVe) chỉ gán **một vector duy nhất** cho mỗi từ, bất kể ngữ cảnh.

Trong Word2Vec:

Vector của "bank" trong "I went to the **bank**"
(tài chính)

GIỐNG HẾT

Vector của "bank" trong "I sat on the **bank**" (bờ sông).

-> Đây là một hạn chế lớn vì nó bỏ qua tính đa nghĩa (Polysemy).

Giải pháp: Mô hình Ngữ cảnh (BERT)

Các mô hình Transformer (như **BERT**, GPT) tạo ra các vector biểu diễn **theo ngữ cảnh**.

Vector của "bank" trong hai câu trên sẽ **HOÀN TOÀN KHÁC NHAU**.

Ý nghĩa thực tế: Đây chính là công nghệ đã giải quyết bài toán WSD (6.1.3) một cách hiệu quả và là nền tảng cho hầu hết các mô hình SOTA NLP hiện nay.

6.2. Ứng dụng của Biểu diễn Ngữ nghĩa

Hiểu văn bản sâu

Các mô hình như BERT (sử dụng biểu diễn ngữ cảnh) có thể hiểu được sự tinh tế, châm biếm, và các mối quan hệ phức tạp trong câu.



Trả lời câu hỏi (QA)

Các hệ thống QA hiện đại (như Google Search) hiểu được ý nghĩa câu hỏi và đoạn văn để tìm câu trả lời chính xác, thay vì chỉ khớp từ khóa.



Tóm tắt văn bản

Mô hình có thể "đọc" một văn bản dài và tạo ra một tóm tắt ngắn gọn nhưng vẫn giữ được ý nghĩa cốt lõi (nhờ hiểu được các vai trò và khái niệm).

Phân tích quan điểm (Sentiment Analysis)

6.3.1. Chuẩn bị Dữ liệu: Thu thập & Gán nhãn



Thu thập dữ liệu (Data Collection):

Nguồn: Mạng xã hội (Twitter, Facebook), Trang đánh giá sản phẩm/phim (Amazon, IMDb, Tiki), Khảo sát khách hàng.

Phương pháp: APIs (Twitter API) hoặc Cào dữ liệu (Web Scraping).



Gán nhãn dữ liệu (Data Labeling):

Cần thiết cho Học có giám sát (Supervised Learning).

Các nhãn phổ biến: **Positive (Tích cực)**, **Negative (Tiêu cực)**, **Neutral (Trung tính)**.

Dữ liệu "Proxy" (Gián tiếp): Sử dụng đánh giá sao (star rating). Ví dụ: 1-2 sao = Negative, 4-5 sao = Positive. (Nhanh, nhưng kém chính xác).

Gán nhãn thủ công: Tốn kém, nhưng cho chất lượng cao nhất.

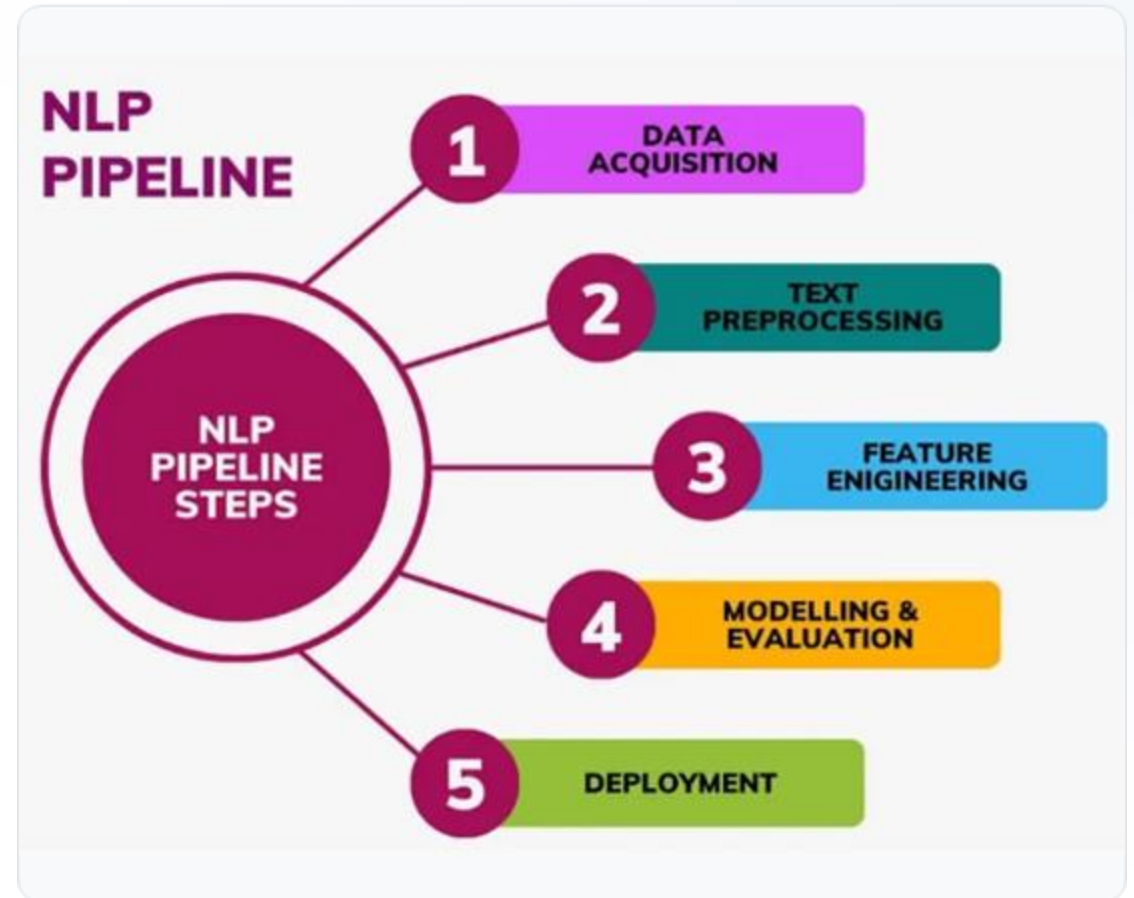
6.3.1. Chuẩn bị Dữ liệu: Tiền xử lý (Preprocessing)

Làm sạch "nhiều" (Noise)

Dữ liệu thô từ mạng xã hội rất "nhiều". Tiền xử lý là bước tối quan trọng để mô hình hoạt động hiệu quả.

Các bước phổ biến:

- ✓ Chuyển về chữ thường (Lowercase).
- ✓ Loại bỏ HTML tags, URLs.
- ✓ Loại bỏ tên riêng (@mentions), hashtags (#).
- ✓ Loại bỏ dấu câu, số (tùy bài toán).
- ✓ **Tokenization:** Tách câu thành các từ (tokens).
- ✓ **Loại bỏ Stop words:** (ví dụ: 'a', 'the', 'is', 'là', 'của'...).
- ✓ **Lemmatization/Stemming:** Đưa từ về dạng gốc.



6.3.1. Chuẩn bị Dữ liệu: Phân chia (Splitting)



Training Set (Tập huấn luyện)

(~70–80%)

Phần dữ liệu lớn nhất, được dùng để "dạy" (huấn luyện) mô hình học máy. Mô hình sẽ học các mẫu (patterns) từ tập này.



Validation Set (Tập kiểm thử)

(~10–15%)

Dùng để "tinh chỉnh" (tune) các tham số (hyperparameters) của mô hình. Ví dụ: nên dùng Naive Bayes hay SVM? Nên dùng 1000 hay 2000 đặc trưng?



Test Set (Tập thử nghiệm)

(~10–15%)

Phần dữ liệu "cấm", mô hình **không bao giờ** thấy trong quá trình huấn luyện/tinh chỉnh. Dùng để đánh giá độ chính xác cuối cùng của mô hình.

6.3.2. Trích chọn Đặc trưng

- Bag of words (BoW)
- TF-IDF (Term Frequency - Inverse Document Frequency).
- N-gram
- Word Embeddings

6.3.2. Trích chọn Đặc trưng: N-grams

Vấn đề của BoW/TF-IDF

Các mô hình này **mất thông tin về thứ tự từ**. Điều này rất tệ cho Phân tích quan điểm.

Ví dụ:

Câu 1: "bộ phim này **rất hay**." (Tích cực)

Câu 2: "bộ phim này **không hay**." (Tiêu cực)

Cả hai câu có thể có vector BoW rất giống nhau (cùng chứa 'phim', 'hay'), dẫn đến dự đoán sai.

Giải pháp: N-grams

Sử dụng các cụm từ (thay vì từ đơn) làm đặc trưng.

- ✓ **1-gram (Unigram):** {phim, hay, không, rất}
- ✓ **2-gram (Bigram):** {phim hay, hay lắm, không hay, rất hay}
- ✓ **3-gram (Trigram):** {phim rất hay, phim không hay}

Ý nghĩa: Bằng cách dùng N-grams (thường là 1+2 gram), mô hình có thể "thấy" được cụm "không hay" và học rằng đây là đặc trưng mạnh của nhãn Tiêu cực.

6.3.2. Trích chọn Đặc trưng: Embeddings

Đặc trưng Học sâu (Deep Learning Features).



Word Embeddings (Word2Vec/GloVe):

Thay vì vector thưa (sparse) của TF-IDF, ta dùng vector đặc (dense) (ví dụ 300 chiều).

Các từ "hay", "tốt", "tuyệt vời" sẽ có vector gần nhau.

Thường lấy vector trung bình (Average) của tất cả các từ trong câu.



Contextual Embeddings (BERT):

Sử dụng vector biểu diễn từ một mô hình Transformer (như BERT) đã huấn luyện trước.

Đây là phương pháp SOTA, vì vector này nắm bắt ngữ nghĩa và ngữ cảnh sâu.

Không cần dùng N-grams vì mô hình tự hiểu thứ tự từ.



Ý nghĩa: Chuyển từ đếm từ (BoW) sang hiểu nghĩa (Embeddings).

6.3.3. Học có giám sát – Khái niệm

 **Khái niệm:** "Dạy" mô hình bằng cách cho nó xem các ví dụ đã được gán nhãn (labeled data).

 **Quy trình (Workflow):**

1 **Input (X):** Văn bản (ví dụ: "phim này rất hay")

2 **Feature Extraction:** Chuyển X thành Vector (ví dụ: TF-IDF của [phim, rất, hay])

3 **Label (y):** Nhãn thật (ví dụ: Positive)

4 **Model:** Mô hình (ví dụ: SVM) "học" mối liên hệ giữa Vector và Label.

 **Dự đoán (Inference):** Khi có văn bản mới (chưa gán nhãn), mô hình sẽ chuyển nó thành vector và dự đoán nhãn.

✓ **Ưu điểm:** Thường cho độ chính xác cao nhất nếu có đủ dữ liệu gán nhãn.



Naive Bayes (NB)

Mô hình dựa trên xác suất
(Định lý Bayes).

Ý tưởng: "Từ 'tuyệt vời'
(wonderful) xuất hiện với xác
suất bao nhiêu % trong các
review Tích cực so với Tiêu
cực?"

Ưu điểm: Rất nhanh, đơn
giản, hiệu quả cho văn bản
(baseline mạnh).



Logistic Regression (LR)

Một mô hình tuyến tính
(linear model) dùng cho bài
toán phân loại.

Ưu điểm: Nhanh, dễ diễn giải
(có thể thấy được đặc trưng
nào 'nặng' nhất).



Support Vector Machines (SVM)

Tìm một "siêu phẳng"
(hyperplane) để phân chia rõ
ràng nhất các điểm dữ liệu
(vector) Tích cực và Tiêu
cực.

Ưu điểm: Rất hiệu quả với
không gian đặc trưng lớn,
nhiều chiều (như TF-IDF + N-
grams).

6.3.3. Học có giám sát – Thuật toán Học sâu

Xử lý văn bản như một chuỗi

Các mô hình học sâu (Deep Learning) không cần bước trích chọn đặc trưng TF-IDF. Chúng học trực tiếp từ các từ (hoặc word embeddings).

- ✓ **Recurrent Neural Networks (RNN/LSTM):**
 - ✓ Đọc văn bản theo thứ tự (tuần tự), từ trái sang phải.
 - ✓ Có "bộ nhớ" để ghi nhớ các từ trước đó.
 - ✓ Rất phù hợp để nắm bắt ngữ cảnh và thứ tự từ (ví dụ: hiểu 'not' đứng trước 'good').
- ✓ **Convolutional Neural Networks (CNN):**
 - ✓ Thường dùng cho ảnh, nhưng trong NLP, nó hoạt động như một bộ phát hiện N-grams hiệu quả.

6.3.3. Học có giám sát – Transformers (SOTA)



Mô hình Transformers (BERT, RoBERTa, GPT).



Khái niệm: Là các mô hình được huấn luyện trước (Pre-trained) trên một lượng dữ liệu văn bản khổng lồ (ví dụ: toàn bộ Wikipedia).



Cơ chế "Attention": Cho phép mô hình nhìn vào *toàn bộ* câu cùng lúc (thay vì tuần tự như LSTM) và quyết định từ nào quan trọng nhất để hiểu ngữ cảnh.



Fine-tuning (Tinh chỉnh):

1

Lấy mô hình BERT (đã hiểu ngôn ngữ chung).

2

"Dạy" thêm (tinh chỉnh) trên dữ liệu gán nhãn của chúng ta (ví dụ: 10,000 review phim).



Ý nghĩa: Đây là phương pháp State-of-the-art (SOTA), cho độ chính xác cao nhất hiện nay, đặc biệt trong việc hiểu ngữ cảnh, châm biếm, và phủ định.

6.3.4. Không giám sát - Lexicon-based

Không cần Dữ liệu gán nhãn

Phương pháp này **không cần** tập Training đã gán nhãn.

Nó hoạt động dựa trên một "Từ điển" (Lexicon) các từ đã được gán điểm quan điểm từ trước.

Ví dụ Lexicon:

- ✓ good, great, wonderful: +1 (Tích cực)
- ✓ bad, poor, terrible: -1 (Tiêu cực)
- ✓ amazing: +2 (Tích cực mạnh)
- ✓ awful: -2 (Tiêu cực mạnh)

Cách thức hoạt động

Đếm điểm của các từ trong câu.

Câu: "This movie is **good** (+1) but the acting is **terrible** (-1)."

Tổng điểm: (+1) + (-1) = **0 (Neutral)**

Câu: "This movie is **amazing** (+2)."

Tổng điểm: +2 (**Positive**)

Ưu điểm: Rất nhanh, dễ triển khai, không cần gán nhãn.

6.3.4. Lexicon-based – Thách thức

Phủ định (Negation):

"This movie is **not good**."

Phương pháp đơn giản sẽ đếm 'good' (+1) và dự đoán sai. Cần các luật để đảo ngược điểm (ví dụ: not good = -1).

Khuếch đại (Intensifiers):

"This movie is **very good**."

Cần luật để tăng trọng số (ví dụ: very good = +1.5 thay vì +1).

Châm biếm (Sarcasm):

"**Great**. My phone just broke."

Từ 'Great' (+1) nhưng ý nghĩa là Tiêu cực. Rất khó để phát hiện.

Từ vựng chuyên ngành (Domain-specific):

Từ "lạnh" (cold) là Tiêu cực (review đồ ăn) nhưng Trung tính (review thời tiết).

6.3.5. Đánh giá Mô hình: Confusion Matrix

Đo lường hiệu suất trên Tập Test

Sau khi huấn luyện, ta dùng mô hình để dự đoán trên tập Test (dữ liệu mô hình chưa thấy).

Ma trận nhầm lẫn (Confusion Matrix) so sánh kết quả **Dự đoán (Predicted)** với kết quả **Thực tế (Actual)**.

(Ví dụ cho bài toán 2 lớp: Positive vs. Negative)

- ✓ **True Positive (TP):** Thực tế là Pos, đoán là Pos (Đúng).
- ✓ **True Negative (TN):** Thực tế là Neg, đoán là Neg (Đúng).
- ✓ **False Positive (FP):** Thực tế là Neg, đoán là Pos (Sai – Lỗi Type I).
- ✓ **False Negative (FN):** Thực tế là Pos, đoán là Neg (Sai – Lỗi Type II).

6.3.5. Đánh giá Mô hình: Các độ đo (Metrics)

- Accuracy
- Precision
- Recall
- F-score

6.3.5. Đánh giá: F1-Score & Mất cân bằng



F1-Score (Trung bình điều hòa):

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

Là một chỉ số duy nhất kết hợp cả Precision và Recall.

Rất hữu ích khi cần cân bằng cả hai, hoặc khi dữ liệu bị mất cân bằng.



Vấn đề Dữ liệu mất cân bằng (Imbalanced Data):

Kịch bản: Một tập dữ liệu có 99% review là Tích cực (Pos) và 1% là Tiêu cực (Neg).

Lazy model: Một mô hình chỉ đơn giản là "luôn luôn dự đoán Tích cực".

Kết quả: Mô hình này sẽ có **Accuracy = 99%** (rất cao!) nhưng nó hoàn toàn vô dụng, vì nó bỏ lỡ (miss) 100% các review Tiêu cực (Recall = 0).



Ý nghĩa: Khi dữ liệu mất cân bằng, **luôn luôn** sử dụng Precision, Recall, và F1-Score thay vì Accuracy.

6.3.5. Đánh giá: K-Fold Cross-Validation

✂ **Vấn đề của việc chia Train/Test đơn giản:** Kết quả có thể phụ thuộc vào "vận may". Nếu vô tình chia tập Test quá dễ, mô hình sẽ có điểm cao giả tạo.

📁 **Giải pháp: K-Fold Cross-Validation (Kiểm tra chéo K-lần)** (ví dụ: $K=5$)

🔧 **Quy trình:**

1 Gộp tập Train và Validation lại, chia chúng thành 5 phần (Folds) bằng nhau.

2 **Lần 1:** Huấn luyện trên Fold 1,2,3,4. Đánh giá trên Fold 5.

3 **Lần 2:** Huấn luyện trên Fold 1,2,3,5. Đánh giá trên Fold 4.

4 ...

5 **Lần 5:** Huấn luyện trên Fold 2,3,4,5. Đánh giá trên Fold 1.

📋 **Kết quả:** Lấy điểm số (ví dụ: F1) trung bình của cả 5 lần.

⚙ **Ý nghĩa:** Cung cấp một đánh giá khách quan và ổn định hơn về hiệu suất của mô hình, đảm bảo mô hình hoạt động tốt trên tất cả các phần của dữ liệu.